

Chapter 42: Authorization

1. Overview

Authentication answers "who are you?" Authorization answers "what are you allowed to do?" The two are frequently conflated, but they are architecturally distinct concerns with different failure modes. A user can be perfectly authenticated (valid JWT, valid session) and still be forbidden from viewing an admin dashboard, deleting another tenant's data, or approving an invoice above a certain amount.

In an Angular application, authorization shows up in four places:

1. **Structural UI gating** — hiding/disabling buttons, menu items, and sections the current user shouldn't see (`*appHasRole`, `*appHasPermission`).
2. **Route-level guards** — preventing navigation into a route/lazy chunk the user isn't entitled to (`CanMatch`, `CanActivate`).
3. **Dynamic UI composition** — building menus, dashboards, and forms whose *shape* depends on the user's permission set, not just visibility toggles.
4. **Data-layer trust boundaries** — the part everyone forgets: none of the above is a security control. It is UX. The server is the only place authorization can be enforced with any guarantee.

This chapter treats Angular-side authorization as what it actually is: a presentation-layer optimization that must degrade gracefully to "access denied" when (not if) it is bypassed, stale, or wrong — because the API will reject the request anyway if it's implemented correctly.

2. Core Concepts

2.1 RBAC vs Claims/Permission-Based Authorization

Role-Based Access Control (RBAC) assigns users to roles (`Admin`, `Editor`, `Viewer`), and roles map to a fixed set of capabilities. The application checks role membership: `if (user.role === 'Admin')`.

- Pros: simple mental model, easy to reason about, cheap to implement, works well when the permission surface is small and stable.
- Cons: coarse-grained. Adding a new capability ("can approve refunds but not issue them") often means inventing a new role or overloading an existing one. Role explosion is the classic RBAC failure mode — you end up with `Admin`, `SeniorAdmin`, `RegionalAdmin`, `AdminReadOnly`, each a one-off hack.

Claims/Permission-Based Authorization assigns discrete, composable permissions directly to a user (often via roles as a convenience grouping, but the check is against the permission, not the role): `invoices:approve`, `invoices:issue`, `users:delete`. The application checks `if (user.permissions.includes('invoices:approve'))`.

- Pros: fine-grained, composable, maps naturally to REST/GraphQL resource-action pairs, scales to complex domains (multi-tenant SaaS, enterprise systems with custom role definitions per customer).
- Cons: more moving parts — permission catalogs, role→permission mapping tables, potentially larger tokens/claims payloads.

Practical convention: most production systems use both. Roles are a *management convenience* (assign a bundle of permissions in one click), but the actual authorization check in code is always against a permission/claim, never a role string. This decouples "what a role means" (can change per tenant, per customer contract) from "what code path is allowed to execute."

```
// Anti-pattern: role check baked into business logic
if (user.role === 'Admin' || user.role === 'SuperAdmin') { ... }

// Preferred: permission check, role is just how the permission was granted
if (user.hasPermission('invoice:approve')) { ... }
```

2.2 Why Client-Side Authorization Is UX-Only, Never a Security Boundary

This is the single most important idea in this chapter, and the one interviewers probe hardest.

Everything running in the browser is untrusted from the server's point of view:

- The entire Angular bundle is downloadable and inspectable. Route guards, directives, and permission checks are plain JavaScript that any user can read, patch with dev tools, or bypass by calling the API directly (curl, Postman, a script).
- localStorage / sessionStorage / in-memory state holding the user's permission list can be edited by the user in the browser console before any check runs.
- A *appHasPermission directive that hides a "Delete User" button does nothing to stop a request to DELETE /api/users/42 sent directly to the backend.
- A CanMatch guard blocking /admin navigation does nothing to stop someone hitting /api/admin/reports directly.

Therefore:

- **Client-side authorization exists to produce a good user experience:** don't show buttons that will 403, don't let users waste time filling a form they can't submit, reflect their permission set in navigation so the UI feels coherent.
- **Server-side authorization is the only real security boundary.** Every mutating and every sensitive read must be re-checked against the authoritative permission source (database, IdP claims, policy engine) on the server, on every request, independent of anything the client asserted.
- This is "defense in depth" applied to the client/server split: the client is one (soft) layer, the server is the (hard) layer. If you only have the client-side check, you have no security at all — you have cosmetic UI.

A useful interview framing: *"If I disabled JavaScript validation entirely and replayed every network request with a proxy, would the system still be secure? If the answer is no, the authorization is decorative."*

2.3 Hiding vs Restricting

- **Hiding:** UI element is not rendered or is disabled. Purely presentational. Achieved with structural directives (*appHasPermission) or [disabled] bindings.
- **Restricting:** the action is actually prevented — request rejected server-side with 403 Forbidden, route data never sent to the client, feature flags evaluated server-side before payload assembly.

A mature app couples both: hide the button for a clean UX, and rely on the API gateway/backend to reject anything that slips through (stale cache, tampered client, direct API call, race condition after a permission downgrade mid-session).

2.4 Route-Level Authorization Guards

Angular's router supports two relevant guard types:

- `CanActivate` (and `CanActivateChild`) — runs after the route has matched, can redirect. Historically the default place to put "is logged in / has role" checks.
- `CanMatch` (replaces the older `CanLoad` for lazy modules, and can also gate matching of any route, not just lazy ones) — runs *during route matching*, before the route is even selected as a candidate. If it returns `false`, the router continues trying other routes that match the same path, which lets you register two routes at the same path — one for authorized users, one showing a "not authorized" or "not found" fallback — without ever downloading the lazy chunk for the unauthorized path.

`CanMatch` is preferred for authorization today because:

1. It prevents the lazy-loaded bundle from being fetched at all for users without permission (real, if minor, security-by-obscurity benefit — the code isn't even in the network tab).
2. It composes cleanly with multiple routes at the same path for role-based branching.
3. It runs earlier, so no partial navigation/flash occurs.

Authentication and authorization guards should be composed as separate, single-responsibility guards, not one giant guard: an authentication guard establishes *is there a valid session at all*, and one or more authorization guards establish *does this identity have the required claim*. Angular's functional guards (`CanMatchFn`, `CanActivateFn`) make this composition trivial via `combineGuards`-style arrays — the router already ANDs all guards on a route; each guard should do one job and return quickly.

2.5 Dynamic Menu / UI Generation Based on Permissions

Rather than hardcoding menu items and wrapping each one in a structural directive, mature applications build a permission-aware menu model: an array of `{ label, route, requiredPermission }` descriptors is filtered reactively against the current user's permission set to produce the rendered menu. This:

- centralizes the mapping between UI affordances and permissions (auditable, testable in isolation from templates),
- avoids permission-check logic scattered across dozens of templates,
- allows the same descriptor list to drive both the sidebar and route guards (single source of truth), reducing the chance that a route is reachable but not in any menu, or listed in the menu but not actually guarded.

2.6 Combining Authentication State with Authorization Checks

Authorization checks are meaningless without a settled authentication state. Two race conditions must be handled:

1. **Guard ordering:** an authorization guard must never run before authentication state has resolved (e.g., before a "who am I" call returns). Guards should `await` /return an observable that only emits once auth

state is known, otherwise you get false negatives (denying access to a legitimate user because permissions hadn't loaded yet).

2. **UI flicker:** templates gating on permissions must not render the "denied" or the "granted" branch until the permission signal has an initial value — otherwise you get a flash of privileged UI (security-relevant leak, however brief) or a flash of "access denied" for a user who is actually authorized (bad UX). The pattern is a three-state signal: `loading | granted | denied`, not a two-state boolean defaulting to `false`.

2.7 Multi-Tenant Authorization Considerations

In multi-tenant SaaS, authorization has an extra dimension: permissions are scoped *per tenant*, not global to the user.

- A user's permission set must always be evaluated in the context of "permission X, within tenant Y" — `hasPermission('invoice:approve', tenantId)`, not a flat global list. A user who is `Admin` in Tenant A must not implicitly be `Admin` in Tenant B.
- Tenant switching (a user with access to multiple tenants switching context in the UI) must invalidate and reload the entire permission/claims cache — a stale permission set from the previous tenant is a cross-tenant privilege leak.
- The server must independently derive tenant context from the authenticated session/token (not trust a `tenantId` query param or client-asserted header) to prevent horizontal privilege escalation (Tenant A user requesting Tenant B's resource by guessing an ID).
- Route guards in multi-tenant apps typically need the resolved tenant in scope before the permission check runs, which means tenant resolution becomes another upstream guard/resolver in the chain: `authGuard → tenantResolver → permissionGuard`.

3. Code Examples

3.1 `PermissionsService` — Reactive Permission State with Signals

```
// permissions.service.ts
import { Injectable, computed, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';

export interface PermissionSet {
  tenantId: string;
  permissions: string[]; // e.g. ['invoice:approve', 'user:delete']
}

type PermissionState =
  | { status: 'loading' }
  | { status: 'loaded'; permissions: Set<string>; tenantId: string }
  | { status: 'error' };

@Injectable({ providedIn: 'root' })
export class PermissionsService {
  private readonly state = signal<PermissionState>({ status: 'loading' });
```

```

/** True only once permissions have actually loaded – avoids flicker. */
readonly isReady = computed(() => this.state().status === 'loaded');

readonly currentTenantId = computed(() => {
  const s = this.state();
  return s.status === 'loaded' ? s.tenantId : null;
});

constructor(private readonly http: HttpClient) {}

async loadForCurrentSession(): Promise<void> {
  this.state.set({ status: 'loading' });
  try {
    const result = await firstValueFrom(
      this.http.get<PermissionSet>('/api/me/permissions'),
    );
    this.state.set({
      status: 'loaded',
      permissions: new Set(result.permissions),
      tenantId: result.tenantId,
    });
  } catch {
    this.state.set({ status: 'error' });
  }
}

/** Call whenever the user switches tenant context – never reuse stale permissions. */
async switchTenant(tenantId: string): Promise<void> {
  this.state.set({ status: 'loading' });
  await this.loadForCurrentSession();
  const s = this.state();
  if (s.status === 'loaded' && s.tenantId !== tenantId) {
    // Defensive check: server disagreed with requested tenant – treat as error, not
    silent success.
    this.state.set({ status: 'error' });
  }
}

/**
 * Synchronous, three-state-aware check for use in computed()/effects.
 * Callers that need to distinguish "still loading" from "denied" should
 * read `isReady()` first rather than relying on this returning false during load.
 */
has(permission: string): boolean {
  const s = this.state();
  return s.status === 'loaded' && s.permissions.has(permission);
}

hasAny(permissions: string[]): boolean {
  return permissions.some((p) => this.has(p));
}

```

```

hasAll(permissions: string[]): boolean {
    return permissions.every((p) => this.has(p));
}
}

```

3.2 *appHasPermission Structural Directive

```

// has-permission.directive.ts
import {
    Directive,
    Input,
    TemplateRef,
    ViewContainerRef,
    effect,
    inject,
} from '@angular/core';
import { PermissionsService } from '../permissions.service';

/**
 * Structural directive: *appHasPermission="invoice:approve"
 * Also supports arrays for "any of": *appHasPermission="['a','b']"
 *
 * Renders nothing while permissions are still loading (avoids flicker of
 * privileged content), then reactively shows/hides as permission state changes
 * (e.g. after a tenant switch) without requiring a page reload.
 */
@Directive({
    selector: '[appHasPermission]',
    standalone: true,
})
export class HasPermissionDirective {
    private readonly templateRef = inject(TemplateRef<unknown>);
    private readonly viewContainer = inject(ViewContainerRef);
    private readonly permissions = inject(PermissionsService);

    private required: string[] = [];
    private hasView = false;

    @Input() set appHasPermission(value: string | string[]) {
        this.required = Array.isArray(value) ? value : [value];
    }

    constructor() {
        // effect() re-runs whenever the underlying permission signal changes,
        // e.g. after loadForCurrentSession() or switchTenant() resolve.
        effect(() => {
            const ready = this.permissions.isReady();
            const allowed = ready && this.permissions.hasAny(this.required);

            if (allowed && !this.hasView) {
                this.viewContainer.createEmbeddedView(this.templateRef);
                this.hasView = true;
            }
        });
    }
}

```

```

    } else if (!allowed && this.hasview) {
      this.viewContainer.clear();
      this.hasView = false;
    }
    // Note: while `ready` is false, `allowed` is false too, so nothing
    // renders yet – this is the deliberate anti-flicker behavior.
  });
}
}

```

Usage:

```

<button *appHasPermission="'invoice:approve'" (click)="approve(invoice)">
  Approve
</button>

<section *appHasPermission="['reports:view', 'reports:export']">
  <!-- shown if the user has EITHER permission -->
</section>

```

3.3 Role/Permission-Based Route Guard Using CanMatch

```

// permission.guard.ts
import { inject } from '@angular/core';
import { CanMatchFn, Route, Router, UrlSegment } from '@angular/router';
import { PermissionsService } from './permissions.service';
import { firstValueFrom, filter, take } from 'rxjs';
import { toObservable } from '@angular/core/rxjs-interop';

/**
 * Factory that produces a CanMatchFn requiring a specific permission.
 * Use per-route via route.data, e.g.:
 * { path: 'admin', canMatch: [permissionGuard], data: { permission: 'admin:access' } }
 */
export const permissionGuard: CanMatchFn = async (route: Route, segments: UrlSegment[]) => {
  const permissions = inject(PermissionsService);
  const router = inject(Router);
  const required = route.data?.['permission'] as string | undefined;

  if (!required) {
    // Misconfiguration: a guarded route MUST declare what it requires.
    // Fail closed, not open.
    return false;
  }

  // wait for permission state to settle (avoids the race where this guard
  // runs before the initial /api/me/permissions call resolves).
  await firstValueFrom(
    toObservable(permissions.isReady).pipe(filter(Boolean), take(1)),
  );

  if (permissions.has(required)) {

```

```

    return true;
  }

  // Returning a UrlTree redirects instead of just failing the match,
  // giving the user an explicit "forbidden" page rather than a silent 404.
  return router.parseUrl('/forbidden');
};

```

```

// app.routes.ts – composing authentication + authorization guards
import { Routes } from '@angular/router';
import { authGuard } from './auth.guard';
import { permissionGuard } from './permission.guard';

export const routes: Routes = [
  {
    path: 'admin',
    canMatch: [authGuard, permissionGuard], // authentication first, then authorization
    data: { permission: 'admin:access' },
    loadChildren: () => import('./admin/admin.routes').then((m) => m.ADMIN_ROUTES),
  },
  {
    path: 'admin',
    // Fallback route matched when the guards above return false/UrlTree –
    // CanMatch lets the router try the next route at the same path.
    component: NotAuthorizedComponent,
  },
];

```

3.4 Dynamic Sidebar Menu Filtered by Permissions

```

// menu.model.ts
export interface MenuItem {
  label: string;
  route: string;
  icon: string;
  requiredPermission?: string; // absent = visible to any authenticated user
  children?: MenuItem[];
}

export const MENU_DEFINITION: MenuItem[] = [
  { label: 'Dashboard', route: '/dashboard', icon: 'home' },
  {
    label: 'Invoices',
    route: '/invoices',
    icon: 'file-invoice',
    requiredPermission: 'invoice:view',
    children: [
      { label: 'Approve', route: '/invoices/approve', icon: 'check', requiredPermission:
'invoice:approve' },
      { label: 'Issue', route: '/invoices/issue', icon: 'send', requiredPermission:
'invoice:issue' },
    ],
  },
];

```

```

    },
    {
      label: 'Administration',
      route: '/admin',
      icon: 'shield',
      requiredPermission: 'admin:access',
      children: [
        { label: 'Users', route: '/admin/users', icon: 'users', requiredPermission:
'admin:manage' },
        { label: 'Tenants', route: '/admin/tenants', icon: 'building', requiredPermission:
'admin:manage' },
      ],
    },
  ],
};

```

```

// sidebar-menu.component.ts
import { ChangeDetectionStrategy, Component, computed, inject } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { PermissionsService } from './permissions.service';
import { MENU_DEFINITION, MenuItem } from './menu.model';

@Component({
  selector: 'app-sidebar-menu',
  standalone: true,
  imports: [RouterLink, RouterLinkActive],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (permissions.isReady()) {
      <nav>
        @for (item of visibleMenu(); track item.route) {
          <a [routerLink]="item.route" routerLinkActive="active">
            <i [class]="icon" + item.icon"></i> {{ item.label }}
          </a>
          @if (item.children?.length) {
            <div class="submenu">
              @for (child of item.children; track child.route) {
                <a [routerLink]="child.route" routerLinkActive="active">{{ child.label }}
              </a>
            </div>
          }
        }
      </nav>
    } @else {
      <nav class="skeleton" aria-hidden="true"><!-- loading placeholder, no flicker -->
    </nav>
  }
  `
})
export class SidebarMenuComponent {
  protected readonly permissions = inject(PermissionsService);
}

```

```

/**
 * Recomputes automatically whenever permission state changes (tenant switch,
 * role update, re-login). A top-level item is shown only if the user can see
 * it AND has at least one visible child (if it has children at all) –
 * prevents a parent nav entry that expands to nothing.
 */
protected readonly visibleMenu = computed<MenuItem[]>(() =>
  this.filterMenu(MENU_DEFINITION),
);

private filterMenu(items: MenuItem[]): MenuItem[] {
  return items
    .filter((item) => !item.requiredPermission ||
this.permissions.has(item.requiredPermission))
    .map((item) => ({
      ...item,
      children: item.children ? this.filterMenu(item.children) : undefined,
    }))
    .filter((item) => !item.children || item.children.length > 0 ||
!item.requiredPermission);
}
}

```

4. Internal Working

4.1 How a Structural Directive Conditionally Renders

A structural directive like `*appHasPermission` desugars (via Angular's template micro-syntax) into an `<ng-template>` with the directive applied, receiving an injected `TemplateRef` (a factory for the embedded view's DOM/bindings) and a `ViewContainerRef` (the logical location in the DOM tree where views can be inserted/removed). The directive itself does no rendering — it decides, based on injected reactive state (a signal or observable from `PermissionsService`), whether to call `viewContainer.createEmbeddedView(templateRef)` (mount) or `viewContainer.clear()` (unmount).

Using Angular's `effect()` (or, pre-signals, an `Subscription` to an observable inside `ngOnInit/ngonDestroy`) means the directive re-evaluates automatically whenever the injected permission state changes — no manual change detection wiring needed, and the view is fully torn down (not just CSS-hidden) when access is revoked, which matters because a torn-down view cannot leak data through the DOM inspector, whereas `[hidden]` or `display:none` would still leave the (possibly sensitive) DOM node present and readable.

Note the important nuance: destroying the DOM node is a UX/defense-in-depth nicety (it doesn't rely on the user "playing along" with CSS), but the *data* backing that node must not have been fetched into the browser in the first place if it's genuinely sensitive — a torn-down template still executed change detection against data that was already sitting in memory. Real protection means the server never sends the sensitive payload to a caller lacking the permission.

4.2 How Route Guards Compose

The Angular Router evaluates all guards attached to a route (and its ancestors, for `canActivateChild`) and effectively ANDs the results: navigation proceeds only if every guard resolves to `true` (or a redirect `UrlTree`, which short-circuits the rest and triggers a redirected navigation instead).

For `canMatch` specifically: guards run *during* the route-matching algorithm, before a route config is settled on as "the" match for a given URL. If any `canMatch` guard returns `false` for a candidate route, the router does not treat it as an error — it continues to the *next* route definition in the array that also matches the same path pattern. This is what enables the "same path, two route definitions" pattern used in the code example above (`admin` guarded route + `admin` fallback route). If a `canMatch` guard returns a `UrlTree`, the router redirects immediately rather than falling through.

Guard functions can be `async` (return a `Promise<boolean | UrlTree>`) or return an `Observable`; the router awaits/subscribes and uses the first emitted value, then cancels further work — this is why authorization guards should be structured to resolve exactly once permission state is settled (`filter(Boolean)`, `take(1)` pattern) rather than staying subscribed indefinitely.

Ordering matters: listing `authGuard` before `permissionGuard` in the `canMatch` array is a convention, not an enforced dependency — the router runs guards and, depending on Angular version, may run them concurrently or in array order but wait on all before combining results (check current Angular docs for exact concurrency semantics per version; treat guard independence as the safe assumption and have each guard itself verify prerequisite state, e.g., the permission guard should tolerate being asked before auth resolves and fail closed rather than assume auth already ran).

4.3 Why Authorization Must Be Re-Validated Server-Side, Every Request

HTTP is stateless per request from the server's perspective (even with cookies/sessions, the server re-derives identity and permissions on each incoming request from the trusted session store/JWT signature — it does not remember "I already approved this browser tab five minutes ago" as a substitute for checking). Consequences:

- A token/session can be valid at request N and have its underlying role/permission revoked at request N+1 (admin demotes the user mid-session). If the server trusted a client-cached permission claim instead of re-deriving from the authoritative source (or at minimum a short-TTL, server-controlled cache), the revoked user would retain access until token expiry.
- Any client state (headers, request bodies, even signed-looking JWT claims if the server doesn't verify the signature and expiry correctly) can be forged or replayed by a sufficiently motivated actor with tools like Burp/Postman that bypass the Angular app entirely.
- Defense in depth means each layer assumes the layer(s) in front of it may have failed or been bypassed: the browser UI hid the button (layer 1), the router guard blocked navigation (layer 2), but layer 3 — the API's authorization middleware/policy check against the database or IdP — is the one that actually can't be bypassed by a client, because it runs in an environment the client doesn't control.

5. Edge Cases & Gotchas

- **False sense of security from client-only checks.** The most common real-world mistake: a team implements `*appHasPermission` and a `CanMatch` guard, sees the UI behave correctly, and never adds

server-side checks because "the button's already hidden." Any direct API call (or a user opening dev tools and manually invoking a service method bound to a still-loaded component) bypasses all of it. Always pen-test/verify by hitting endpoints directly, ignoring the UI.

- **Stale permission caching after role/permission changes.** If `PermissionsService` loads permissions once at login and caches them for the session, an admin revoking a permission mid-session has no effect until the user refreshes or re-authenticates. Mitigations: short client cache TTL with periodic re-fetch, server-pushed invalidation (WebSocket/SSE event on permission change forcing a client re-fetch), or re-fetching permissions on sensitive-route entry rather than trusting the in-memory cache indefinitely. The server-side check remains the actual safety net regardless of client staleness.
 - **Flicker of unauthorized UI before permission data loads.** If the directive/guard defaults to `false` while loading (rather than a distinct `loading` state), you get one of two bad outcomes: (a) briefly rendering restricted content because the default was `true`/permissive, or (b) briefly showing "access denied"/empty state for an authorized user because the default was `false`, causing a jarring flash-then-appear. The fix, shown in the code above, is a tri-state model (`loading | granted | denied`) and gating render on `isReady()` before evaluating the permission at all.
 - **Permission checks scattered across templates.** `*ngIf="user.role === 'Admin' || user.role === 'Manager' || user.permissions.includes('x')"` duplicated across dozens of templates is unmaintainable and error-prone (one template gets the OR logic wrong, silently over- or under-exposing UI). Centralize the check inside `PermissionsService` (`hasAny`, `hasAll`) and inside declarative structures like the `MENU_DEFINITION` array so there is exactly one place that encodes "what permission does this feature need."
 - **Guard/permission mismatch drift.** The set of permissions checked in route guards, the set checked in structural directives, and the set encoded in the menu definition can silently diverge over time (e.g., someone updates the guard's required permission but forgets the menu descriptor). Where possible, derive all three from one shared source of truth (e.g., route `data.permission` read by both the guard and a menu-builder that walks the route config) instead of hand-maintaining three parallel lists.
 - **Tenant-switch permission leakage.** Forgetting to fully clear/reload `PermissionsService` state on tenant switch can let a user retain Tenant A's elevated permissions while the UI context has moved to Tenant B, if the service only replaces individual fields instead of resetting to `loading` first (as shown in `switchTenant()` above).
 - **CanMatch returning false silently 404ing.** If no fallback route exists at the same path, an unauthorized user simply gets router's default "no match" behavior, which can look like a broken link rather than an intentional denial. Prefer returning a `urlTree` to an explicit "forbidden" page, or always pair a guarded route with a fallback route at the same path.
 - **Authorization guard running before authentication resolves.** If the permission guard checks `permissions.has(...)` synchronously before the initial permissions fetch (which itself depends on a resolved auth session) completes, it will incorrectly deny a legitimate, freshly-logged-in user. Always await a "settled" signal before making the allow/deny decision (shown via `toObservable(permissions.isReady)` in the guard example).
-

6. Interview Questions & Answers

Q1. What is the difference between authentication and authorization?

Authentication verifies identity ("who are you" — login, token validation). Authorization determines what an authenticated identity is permitted to do ("what can you access"). A request can pass authentication and still fail authorization (valid user, insufficient rights).

Q2. What's the difference between RBAC and permission/claims-based authorization?

RBAC checks membership in a named role (`user.role === 'Admin'`); permission-based authorization checks discrete capabilities (`user.permissions.includes('invoice:approve')`), often with roles used only as a convenient way to *assign bundles* of permissions. Permission-based systems scale better for fine-grained, evolving access models and avoid "role explosion" (inventing a new role for every new access combination).

Q3. Why is client-side (Angular) authorization not a security control?

Interviewer intent: checking whether the candidate understands the trust boundary between client and server, not just how to write a guard.

Everything in the browser — JS bundles, directive logic, route guards, cached permission flags — is fully inspectable and modifiable by the end user (dev tools, proxies like Burp/Postman, direct `curl` calls to the API). No client-side check can prevent a determined actor from calling the backend directly with a forged or replayed request. Client-side authorization only improves UX (hiding buttons/routes that would fail anyway); the server must independently re-verify the caller's rights on every request against an authoritative source, because it's the only environment the client doesn't control.

Q4. How would you implement a structural directive like `*appHasPermission`?

Inject `TemplateRef` and `ViewContainerRef` in the directive constructor, accept the required permission(s) via an `@Input()` setter matching the directive's selector name, and reactively call `viewContainer.createEmbeddedView(templateRef)` or `viewContainer.clear()` based on the current permission state (ideally via `effect()` over a signal-based `PermissionsService`, so it automatically re-renders when permissions change, e.g. after a tenant switch, without manual subscription management).

Q5. Why prefer `CanMatch` over `CanActivate` for authorization on lazy-loaded feature routes?

`CanMatch` runs during the route-matching phase, before the route is selected as a match — so if it returns `false`, the corresponding lazy chunk is never fetched, and the router can fall through to another route registered at the same path (e.g., a "forbidden" fallback). `CanActivate` runs after the route has already been chosen as the match, so while it can still block activation/redirect, it doesn't prevent code-splitting artifacts tied to that route from having been resolved, and doesn't support the "alternate route at same path" pattern as cleanly.

Q6. How do you compose authentication and authorization guards on the same route?

Write them as separate, single-responsibility functional guards and list them together in the route's `canMatch / canActivate` array — Angular ANDs all guards on a route. The authentication guard establishes there is a valid session; authorization guard(s) check specific permissions/roles, typically declared via `route.data` so the same guard function is reusable across many routes (data-driven guard). Each guard should independently tolerate being invoked before upstream state (like auth) has resolved, rather than assuming guard array order guarantees sequential completion.

Q7. What's the risk of caching a user's permission set in the client and not refreshing it?

Interviewer intent: testing awareness of authorization staleness as a real (if secondary) security/UX issue, not just performance.

If an admin revokes a role or permission mid-session, a client that cached the old permission set at login will

continue to show privileged UI (and, if the server also trusts a long-lived unverified claim instead of re-deriving from the authoritative source, may continue to permit privileged actions) until the token expires or the user manually refreshes. Mitigations include short TTL re-fetching, server-pushed invalidation events (WebSocket/SSE) that trigger a client re-fetch, and — most importantly — ensuring the server itself re-validates permissions per request against a source that reflects near-real-time state, rather than trusting a JWT's embedded permission claims for the full lifetime of a long-lived token.

Q8. How do you avoid a flash of unauthorized/authorized content while permissions are loading?

Model permission state as three states (`Loading` | `granted` | `denied`), not a boolean defaulting to `false` or `true`. Gate rendering on an `isReady()` signal first; only evaluate and render the granted/denied branch once the initial fetch has resolved. This prevents both a security-relevant flash of privileged content (if defaulting permissive) and a jarring UX flash of "denied" for legitimate users (if defaulting to deny before data arrives).

Q9. How would you build a dynamic sidebar menu that reflects the current user's permissions?

Define menu items declaratively as a data structure (`{ label, route, requiredPermission, children }`) separate from the rendering template. Use a `computed()` /pure function that recursively filters the tree against `PermissionsService`, dropping items whose required permission is absent and dropping parent items left with no visible children. This centralizes permission-to-UI mapping in one auditable place instead of scattering `*ngIf` permission checks across many templates, and it recomputes reactively whenever the underlying permission signal changes (e.g., tenant switch).

Q10. In a multi-tenant application, why can't you just check `user.permissions.includes(...)` globally?

Interviewer intent: checking whether the candidate has thought beyond single-tenant RBAC/ABAC into cross-tenant isolation, a common real-world gap.

Because permissions in a multi-tenant system are scoped to a tenant — a user who is `Admin` for Tenant A must not implicitly be treated as `Admin` for Tenant B. Checks must be parameterized by tenant context (`hasPermission(permission, tenantId)`), the tenant context itself must be derived server-side from the authenticated session (never trusted from a client-supplied `tenantId`), and switching tenant context client-side must fully invalidate and reload the permission cache rather than patch it, to prevent a residual, stale permission from a previous tenant leaking into the new context.

Q11. What happens if a `canMatch` guard returns `false` and there's no fallback route at the same path?

The router treats it as "no route matched," which typically results in whatever the app's wildcard (`**`) route is configured to do (often a generic 404/not-found page), which can be confusing for users who expected an explicit "you don't have permission" message. Best practice is either to return a `urlTree` redirecting to a dedicated forbidden page, or to register an explicit fallback route at the identical path so `canMatch`'s "try the next matching route" behavior produces a deliberate, informative result.

Q12. How do you prevent permission-check logic from becoming unmaintainable as the app grows?

Centralize all permission evaluation logic behind a single service (`PermissionsService.has/hasAny/hasAll`) so templates and guards call a semantic method rather than re-deriving boolean logic ad hoc. Drive UI structure (menus, forms, feature flags) from declarative data (permission-annotated config objects) rather than imperative `*ngIf` chains repeated per template. Where feasible, derive route guards' required permissions and menu visibility from the same route configuration (`route.data.permission`) so there is one source of truth instead of three independently maintained lists that can drift out of sync.

Q13. Is it acceptable to put the authorization check only in the API and skip it entirely in Angular?

Functionally, yes — security is preserved because the server is authoritative. However, this produces poor UX (users click into pages/actions that will visibly fail, feels broken) and wastes network round-trips on requests that will always be rejected. The recommended approach is both: server enforces (required, non-negotiable), client mirrors the same rules for UX only (optional but expected in production apps), explicitly understanding the client copy is disposable/cosmetic and never a substitute for the server check.

Q14. How would you test authorization logic in an Angular app?

Unit-test `PermissionsService` directly (mock the HTTP response, assert `has` / `hasAny` / `hasAll` behave correctly, including the `loading` / `error` states). Unit-test the structural directive by providing a mock `PermissionsService` via DI and asserting the view container's embedded view count. Unit-test guard functions directly as plain functions (via `TestBed.runInInjectionContext`), asserting they return `true`, `false`, or the expected `UrlTree` for various permission states. Separately, integration/e2e tests should verify that hiding UI is backed by actual server-side 403s when the same restricted action is invoked directly (e.g., via an API test suite), closing the loop between "looks secure" and "is secure."

7. Quick Revision Cheat Sheet

- **Authentication** = who you are. **Authorization** = what you can do. Passing one doesn't imply the other.
- **RBAC**: check role membership (`role === 'Admin'`) — simple, coarse, prone to role explosion.
- **Claims/permission-based**: check discrete capabilities (`permissions.includes('x:y')`) — fine-grained, composable; roles become just a grouping mechanism for assigning permissions.
- **Client-side authorization (directives, guards) = UX only**. It can always be bypassed (dev tools, direct API calls). It is never a security boundary.
- **Server-side authorization = the only real security boundary**. Must re-validate on every request against an authoritative, near-real-time source.
- **Hiding** (don't render/disable) vs **restricting** (server rejects with 403) — both are needed; only the second is security.
- ***appHasPermission directive**: injects `TemplateRef` + `ViewContainerRef`, reactively creates/clears the embedded view based on signal-driven permission state; gate on a `loading` state to avoid flicker.
- `CanMatch` > `CanActivate` for authorization on lazy routes: runs during matching, blocks the lazy chunk from loading, supports "same path, alternate fallback route" pattern; return a `UrlTree` for explicit redirects.
- **Compose guards, don't merge them**: separate authentication guard + authorization guard(s), combined via the route's guard array; each guard should tolerate being invoked before upstream state is settled and fail closed.
- **Dynamic menus**: derive from a declarative, permission-annotated config, filtered reactively — single source of truth, not scattered `*ngIf`s.
- **Stale permissions**: cache with short TTL or invalidate via server push; never trust a long-lived client-held permission claim as the final word.
- **Tenant scoping**: permissions are always (`permission, tenantId`) pairs; full cache reset on tenant switch; tenant context derived server-side, never from client input.
- **Anti-flicker rule**: model permission state as `loading` | `granted` | `denied`, never a boolean defaulting

either way.

- **Golden interview line:** "If someone bypassed the Angular app entirely and hit the API directly, would the system still be secure? If not, the client-side check is decorative, not defense."

Created By - Durgesh Singh